JavaScript Research

While vs Do...While and If vs Switch

This document explains two important pairs of JavaScript control-flow statements, with syntax, examples, outputs, and comparisons.

1. While vs Do...While

Both **while** and **do...while** are loops used to repeat code while a condition is true.

A. While Loop

A **while** loop checks its condition **before** executing the code. If the condition is false at the beginning, the loop body does not run.

```
while (condition) {
    // Code to execute
}
```

Example:

```
let i = 1;

while (i <= 3) {
    console.log(i);
    i++;
}
```

Output: 1, 2, 3

B. Do...While Loop

A **do...while** loop executes the code first and checks the condition afterward. Therefore, the loop body always runs at least once.

```
do {
    // Code to execute
} while (condition);
```

Example:

```
let i = 5;

do {
    console.log(i);
    i++;
} while (i <= 3);
```

Output: 5

The output is 5 because the body executes before JavaScript checks whether i <= 3.

2. Comparison: While vs Do...While

Feature	while	do...while
Condition check	Before execution	After execution
Minimum executions	Zero times	One time

Feature	while	do...while
If condition is initially false	Body does not run	Body runs once
Best use	When the condition must be checked first	When code must run at least once

3. If vs Switch

Both **if...else** and **switch** are conditional statements used to choose which code should run.

A. If...Else Statement

The **if...else** statement runs different code depending on whether a condition is true or false. You can use **else if** to check several conditions.

```
let grade = 85;

if (grade >= 90) {
    console.log("Excellent");
} else if (grade >= 75) {
    console.log("Very Good");
} else {
    console.log("Needs Improvement");
}
```

Output: Very Good

When to use it: Use if...else for ranges, comparisons, or complex conditions, such as checking a student's grade or age.

B. Switch Statement

The **switch** statement compares one expression against several specific values and runs the matching case.

```
let day = 3;

switch (day) {
    case 1:
        console.log("Saturday");
        break;
    case 2:
        console.log("Sunday");
        break;
    case 3:
        console.log("Monday");
        break;
    default:
        console.log("Invalid day");
}
```

Output: Monday

The **break** statement stops execution from continuing into the next case. The **default** block runs if no case matches.

4. Comparison: If vs Switch

Feature	if...else	switch
Conditions	Ranges and complex logic	One expression matched to specific values

Feature	if...else	switch
Comparisons	Supports >, <, >=, ===, and more	Matches case values using strict equality
Readability	Good for different conditions	Good for many fixed choices
Example use	Checking age or grades	Selecting a day or menu option